FinShield Core — Solo Build Roadmap (8 Weeks)

A multi-agent AML case manager and credit risk pipeline: Next.js dashboard + FastAPI gateway + Celery/Redis agent orchestration + Postgres/pgvector.

Guiding principle for a solo 2-month build: get one case flowing end-to-end (even with stub agents) by the end of Week 2, then layer in real intelligence. Never let "polish" block "works."

Week 0 (Pre-work, ~2–3 days): Scaffolding & Decisions

Goal: Repo, environments, and architecture decisions locked so you don't re-architect mid-sprint.

Monorepo layout:

```
/apps/web      (Next.js + Tailwind)
/apps/api      (FastAPI)
/apps/worker   (Celery agents, shares models with api)
/packages/schemas (Pydantic/TS shared types — or just duplicate, keep simple)
/infra         (docker-compose, migrations)
```

`docker-compose.yml`: Postgres (with `pgvector` extension pre-baked via a custom Dockerfile or `ankane/pgvector` image), Redis, FastAPI, Celery worker, Celery beat (optional), Next.js.

Decide agent framework now: **LangGraph over CrewAI** for this use case — you want a deterministic state machine (KYC → Sanctions → Market Risk → Aggregation → Report), not autonomous crew delegation. LangGraph's graph model maps directly to an auditable compliance workflow and makes state persistence/checkpointing much easier to reason about later.

Decide sanctions data source now (this gates Week 4): OFAC SDN list (free, downloadable CSV/XML, updated daily) is the realistic choice for a solo/portfolio build. Commercial APIs (Refinitiv World-Check, Dow Jones) require sales calls and contracts you won't get in 2 months. Plan to ingest OFAC's list yourself and query it locally — this is also more defensible in an audit trail than "we called a black-box API."

Set up Alembic for migrations from day one — compliance systems live and die by traceable schema history.

Deliverable: `docker-compose up` brings up all 5 services with health checks passing.

Week 1: Data Model + API Gateway Skeleton

Goal: Postgres schema and FastAPI CRUD for the core entities, no AI yet.

Core tables:

`entities` (the customer/corporate applicant — name, type, jurisdiction, incorporation docs metadata)

`cases` (case_id, entity_id, case_type: `sanctions_flag` | `loan_application`, status, created_at)

`transactions` (for AML cases — amount, currency, counterparty, timestamp, raw payload jsonb)

`documents` (uploaded KYC docs, S3/local path, doc_type)

`agent_runs` (case_id, agent_name, status, started_at, finished_at, output jsonb) — **this table is your audit trail backbone**

`risk_assessments` (case_id, score, band, rationale, generated_at, model_version)

`regulation_embeddings` (id, source_doc, chunk_text, embedding vector(1536), metadata jsonb) — pgvector table

FastAPI endpoints:

`POST /cases` — create case, kick off orchestration (stub for now: just enqueue a Celery task)

`GET /cases/{id}` — case detail + status

`GET /cases/{id}/agent-runs` — live audit trail (this is what the frontend polls/streams)

`GET /cases/{id}/report` — final compliance report

`POST /entities`, `GET /entities/{id}`

Deliverable: Swagger UI fully functional against a seeded Postgres. You can create a case via curl and see it land in the DB.

Week 2: Celery/Redis Wiring + LangGraph Skeleton (Stub Agents)

Goal: A case submitted via API actually flows through a 3-node graph and writes results back, even if each node just returns canned data. This is your critical "prove the architecture" week.

Celery task `run_case_pipeline(case_id)` invoked from the `POST /cases` endpoint (via `.delay()`).

LangGraph graph with nodes: `kyc_agent`, `sanctions_agent`, `market_risk_agent`, `aggregator_agent`. Each node:

Reads case state from Postgres

Writes a row to `agent_runs` on start and completion (status transitions: `running` → `completed`/`failed`)

Returns structured output (Pydantic model) merged into LangGraph's shared state

Each stub agent just sleeps 2–5 seconds and returns a hardcoded plausible output — you're proving the plumbing, not the intelligence.

Wire up **Server-Sent Events (SSE) or polling** endpoint (`GET /cases/{id}/stream`) so the frontend can watch agent status flip in near-real-time. SSE is simpler than WebSockets for this one-directional use case and works fine through most proxies.

Deliverable: Hit `POST /cases`, watch `agent_runs` rows appear in order, case ends in `completed` with a stub `risk_assessments` row. This is your demo-able skeleton — protect it, don't break it going forward.

Week 3: Frontend Dashboard (Next.js + Tailwind)

Goal: A clean single-page dashboard, not a full app.

`/` — case submission form (entity name, case type, upload docs)

`/cases/[id]` — live workflow view:

Vertical stepper/timeline of agents (KYC → Sanctions → Market Risk → Aggregation), each node showing status (pending/running/done/failed) driven by the SSE/polling endpoint

Right panel: live-updating raw JSON or formatted summary per completed agent

Final state: risk score gauge/badge (Low/Medium/High), download link for the compliance report

Keep state management simple: React Query (TanStack Query) for polling + cache, no Redux needed for a dashboard this scoped.

Tailwind: use a restrained palette (you don't want a loan-risk dashboard looking like a SaaS marketing page) — grays/navy with a single accent color reserved for risk severity (green/amber/red).

Deliverable: You can submit a case in the browser and visually watch the stub pipeline run to completion.

Week 4: Sanctions Auditor Agent (Real Data)

Goal: Replace the sanctions stub with a real, defensible check.

Nightly (or on-deploy) job: download OFAC SDN list, parse into Postgres `sanctions_entries` table (name, aliases, type, program, id).

Sanctions matching is a **fuzzy name matching problem**, not exact match. Use:

`pg_trgm` extension (already in Postgres, pairs naturally with pgvector) for trigram similarity as a fast first pass

Optionally layer a proper library like `rapidfuzz` in the worker for scoring once trigram narrows candidates

Log every candidate match + score into `agent_runs.output` — false positives are expected and **must be shown**, not hidden, since compliance officers need to see what was checked and why something wasn't a hit

Real agent output: list of potential matches with similarity scores, a boolean `requires_manual_review` flag if any score exceeds threshold, and a plain-language rationale string (this can be templated, doesn't need an LLM yet).

Deliverable: Submitting a case for a real (or deliberately test-fixture) sanctioned entity name surfaces a match with a score and rationale in the dashboard.

Week 5: KYC + Market Risk Agents (LLM-Assisted)

Goal: Bring in the LLM for document reasoning and narrative synthesis — this is where the "AI" in AI agent earns its keep.

KYC Agent: given uploaded entity documents (or structured intake form for now — OCR/doc parsing is a rabbit hole, keep scope to structured data unless you have time later), call Claude/GPT via API to:

Flag inconsistencies (e.g., stated business type vs. transaction patterns)

Summarize entity risk factors in plain language

Output structured JSON (use tool calling / structured output mode, not free text you have to regex)

Market Risk Agent: for loan-application cases, pull relevant market data (a free API like Alpha Vantage or FRED for macro indicators is enough — don't build a real-time market data pipeline solo in 2 months) and have the LLM synthesize a short risk narrative referencing the actual numbers pulled, not hallucinated ones. Pass the fetched data into the prompt explicitly.

Critical discipline: **every LLM output that claims a fact must be traceable to data you fetched and included in the prompt.** Store the exact prompt + response in `agent_runs.output` for auditability — this is non-negotiable for a compliance tool, real or portfolio piece.

Deliverable: KYC and Market Risk nodes produce real, data-grounded narrative output, not canned text.

Week 6: pgvector — Regulation Retrieval

Goal: Use pgvector to ground the final report in actual regulatory text instead of the LLM's unverified paraphrase of "the rules."

Ingest a small, real corpus: FinCEN AML guidance PDFs, FATF recommendations, or your jurisdiction's relevant statutes — even 10–20 documents is enough to demonstrate the pattern. Chunk (~500 tokens, some overlap), embed (OpenAI `text-embedding-3-small` or similar), store in `regulation_embeddings`.

Add a retrieval step in the `aggregator_agent`: given the case's risk factors, do a similarity search (`<=>` operator with an IVFFlat or HNSW index on the vector column) to pull the 3–5 most relevant regulation chunks, and

require the report-writing prompt to cite them.
This turns "AI said it's risky" into "AI said it's risky **and here's the specific regulatory clause it's checking against**" — the actual point of the project.
Deliverable: Final compliance report includes a "Regulatory Basis" section with real cited passages, not generic LLM filler.

Week 7: Report Generation, Audit Trail Polish, Hardening

Goal: Make the output artifact and the system itself trustworthy.
Compliance report as a downloadable PDF (or well-formatted HTML → PDF via a lightweight lib) containing: entity summary, KYC findings, sanctions check results (including near-misses), market/credit risk narrative, cited regulations, final risk score + band, and a full timestamped audit trail of every agent run.
Add basic auth (even simple JWT/session — this is a compliance tool, it cannot be unauthenticated even as a demo) and role concept (analyst vs. reviewer) if time allows.
Error handling: what happens when an external API (OFAC download, market data) is down? Agents should fail gracefully into a `needs_manual_review` state, never silently produce a false "clean" result — this is the single most important reliability property for this domain.
Add basic tests: unit tests for the sanctions matcher (known true/false positive fixtures), integration test for one full case pipeline run.
Deliverable: A case that hits a real error path degrades safely and visibly, not silently.

Week 8: Deployment, Demo Polish, Documentation

Goal: Something you can show, deploy, and explain.
Deploy: Railway/Render/Fly.io for the full stack (all support Postgres+Redis+background workers reasonably cheaply), or a single droplet with docker-compose if you want more control. Avoid over-engineering Kubernetes for a solo portfolio project.
Seed the deployed instance with 3–4 realistic demo cases spanning: a clean case, a sanctions near-miss, a high-risk loan applicant, and an error/degraded case — so a viewer sees the range of behavior in under 5 minutes.
README with architecture diagram, explicit disclaimers (this is a demo/portfolio system, not a certified AML compliance product — say this plainly, both because it's true and because it matters in this domain), and setup instructions.
Record a 3–5 minute walkthrough video — for a project like this, a recorded demo often carries more weight than a live link, since reviewers may not want to create test data themselves.
Deliverable: Live deployed link + video walkthrough + clean README.

Cross-Cutting Notes

Scope discipline (the real risk to a solo 2-month timeline):
Skip OCR/document parsing unless Week 7 goes very smoothly — structured intake forms are a reasonable stand-in for "KYC documents."
Skip real-time market data streaming — polling a free API on-demand per case is entirely sufficient.

Skip building your own auth system if a hosted option (Clerk, Auth0 free tier) saves you real days.
Resist adding a 5th/6th agent — four well-audited agents beat eight shallow ones, especially for demoing the audit trail concept.
On the compliance framing: since this is styled as an AML/sanctions tool, keep the README and any public demo explicit that it's an educational/portfolio system and not a substitute for a licensed compliance vendor or legal sign-off — this is good practice for any project modeling a regulated domain, and it also reads as more credible to technical reviewers, not less.
Suggested week-by-week time budget (assuming ~25–30 solo hours/week):

Week	Focus	Risk if slipped
0	Scaffolding	Delays everything — don't skip
1	Data model + API	Foundation, keep tight
2	Orchestration skeleton	**Highest priority to protect** — this is your proof of concept
3	Frontend	Can run partly parallel with Week 4 if ahead of schedule
4	Sanctions agent	Core differentiator, don't cut
5	KYC/Market agents	Can simplify Market Risk narrative if behind
6	pgvector retrieval	Cuttable to "nice to have" only if severely behind — but it's what makes the architecture description true
7	Reporting/hardening	Don't skip error-path handling even under time pressure
8	Deploy/polish	Protect at least 2 days for the demo video